

01 — Foundations of AI & ML

Everything You Need to Know Before Building Models

Section A

Understanding AI & ML

Intelligence, AI, Types of AI,
Traditional vs ML, AI Hierarchy

Section B

Types of Machine Learning

Supervised, Unsupervised,
Reinforcement, Models & Metrics

Section C

How It Works Inside

Neural Networks, Vectors, Math,
Gradient Descent, Training Cycle

What is Intelligence?

The 4 Pillars That Define Intelligence

LEARN

Gain new knowledge from experience

UNDERSTAND

Comprehend context, not just data

DECIDE

Choose the best action among options

ADAPT

Change behaviour when conditions change

Is This Intelligence?

Example	Learn?	Understand?	Decide?	Adapt?	Intelligent?
Cat avoids hot stove after burn	Yes	Yes	Yes	Yes	YES
Calculator doing 2+2	No	No	No	No	NO
Child changes route to avoid bully	Yes	Yes	Yes	Yes	YES
Parrot repeating 'Hello'	Partial	No	No	No	NO
Delivery driver finding shortcut	Yes	Yes	Yes	Yes	YES

KEY INSIGHT

Intelligence is NOT just knowing things.

A calculator KNOWS math but isn't intelligent — it can't learn, understand context, or adapt.

True intelligence requires ALL 4 pillars working together.

This is why building AI is so hard — we need machines that can do all four.

What is Artificial Intelligence?

Making machines that can think, learn, and decide like humans

HUMAN INTELLIGENCE

Brain: 86 billion neurons
Learns from: Life experiences
Understands: Context & emotions
Develops: Over years of growth
Limited by: Energy, time, bias

ARTIFICIAL INTELLIGENCE

Processor: Billions of calculations/sec
Learns from: Data (millions of examples)
Understands: Patterns in numbers
Develops: Over hours of training
Limited by: Data quality & quantity

AI in Your Daily Life

App	AI Feature	What It Does
Instagram	Recommendation Engine	Analyzes your likes, time spent, shares → shows posts you'll engage with
Gmail	Smart Compose + Spam Filter	Predicts next words as you type; blocks 99.9% of spam automatically
Google Maps	Traffic Prediction	Combines GPS data from millions of phones → predicts traffic 30 min ahead
Netflix	Personalization Engine	Analyzes viewing patterns of similar users → recommends shows you'll like

AI = BRAIN, ROBOT = BODY

Common misconception: AI = Robots

AI is the INTELLIGENCE (software) that can think, learn, and decide.

A Robot is just a BODY (hardware) that moves and interacts.

AI without a robot: Gmail spam filter, Netflix recommendations, ChatGPT

Robot without AI: Factory arm doing same motion repeatedly

Robot WITH AI: Self-driving car, Boston Dynamics robot

Most AI has NO physical body — it lives inside apps and servers.

Types of AI: Three Levels

From what we have today to what might come tomorrow

NARROW AI (ANI)

Specialist — ONE task only

CAN: Beat world champion at chess, drive a car, translate languages, detect cancer in X-rays

CAN'T: Do anything outside its specific training — a chess AI can't even play checkers

Examples: Siri, ChatGPT, Tesla Autopilot, Google Translate, Spam filters

Status: This is ALL current AI

GENERAL AI (AGI)

Human-level — ANY intellectual task

CAN (theoretically): Learn any new skill, transfer knowledge between domains, reason abstractly

CAN'T: Exist yet — no one has built this

Would be like: A single AI that can cook, code, write poetry, AND diagnose diseases

Status: 10-50+ years away (debated)

SUPER AI (ASI)

Beyond human — SURPASSES all humans

CAN (theoretically): Outthink every human in every domain simultaneously

CAN'T: Even be properly imagined — by definition beyond our comprehension

Would be like: An intelligence that solves cancer, climate change, and physics in a day

Status: Purely theoretical, may never exist

IMPORTANT: All current AI — including ChatGPT, Claude, Gemini — is Narrow AI.

It seems smart because it's VERY good at language, but it can't truly understand, feel, or generalize. It predicts the next word based on patterns in training data.

Traditional Programming vs Machine Learning

Two fundamentally different approaches to solving problems

TRADITIONAL PROGRAMMING

Human writes explicit RULES + gives DATA → Computer follows rules → ANSWERS

Example: IF email contains 'lottery' AND sender not in contacts → mark as spam

Programmer must think of EVERY possible case. Miss one rule = wrong answer.

Works when: Rules are clear, few exceptions, logic is straightforward

MACHINE LEARNING

Human gives DATA + correct ANSWERS → Computer discovers RULES on its own

Example: Give 10,000 emails labeled spam/not-spam → ML finds patterns humans miss

Computer finds patterns across millions of examples. Gets better with more data.

Works when: Too many rules, patterns are complex, data is abundant

Traditional or ML? — Real Problems

Problem	Approach	Why?
Spam Detection	ML	Spammers constantly change tactics; rules can't keep up. ML adapts to new patterns.
Tax Calculation	Traditional	Tax rules are fixed by law. Clear formulas. No ambiguity.
Face Recognition	ML	Impossible to write rules for every face angle, lighting, age change. ML learns from examples.
Language Translation	ML	Languages have too many exceptions, idioms, context. ML learns from millions of translated texts.

DECISION GUIDE: Which Approach?

- Can you write ALL the rules clearly?
 - YES: Use Traditional Programming
 - NO: Continue to step 2
- Do you have lots of labeled data (examples with answers)?
 - YES: Use Machine Learning
 - NO: Continue to step 3
- Can you collect/create that data?
 - YES: Collect data, then use ML
 - NO: Try Traditional with best-effort rules, or use pre-trained ML models

AI Hierarchy: AI → ML → DL → LLM

Each layer is a SUBSET of the one above it

AI — Artificial Intelligence

Any technique making machines seem intelligent (includes rule-based systems, expert systems, ML, etc.)

ML — Machine Learning

Machines learn from DATA instead of being explicitly programmed with rules

DL — Deep Learning

ML using NEURAL NETWORKS with many layers — learns complex patterns

LLM — Large Language Models

Massive DL models trained on internet-scale text data using Transformer architecture

Which Level? — Real Examples

Spam Filter → ML

Learns patterns from labeled emails (spam/not-spam). Uses algorithms like Naive Bayes. Doesn't need deep networks.

Google Photos — Face Recognition → DL

Uses Convolutional Neural Networks (CNN) with many layers to detect and recognize faces across angles/lighting.

ChatGPT / Claude → LLM

Transformer-based model trained on billions of text documents. Predicts next token. 175B+ parameters.

Chess Engine (Minimax) → AI (not ML)

Uses search trees and rules programmed by humans. No learning from data — pure algorithmic intelligence.

WHY TRANSFORMERS CHANGED EVERYTHING

2017 paper 'Attention Is All You Need' introduced Transformers. Unlike older models that read text word-by-word, Transformers read ALL words at once using 'self-attention'. This made GPT, BERT, and all modern LLMs possible.

Supervised Learning: Classification & Regression

The teacher shows correct answers → model learns the pattern

CLASSIFICATION — Predict a WORD (Category)

Output: A label/category from a fixed set of options

Email → Spam or Not Spam | Image → Cat or Dog or Bird
Tumor → Benign or Malignant | Transaction → Fraud or Legit
Review → Positive or Negative | Loan → Approve or Reject

Binary (2 options) or Multi-class (3+ options)

REGRESSION — Predict a NUMBER (Continuous value)

Output: A specific number on a continuous scale

House features → Price (₹45.7 lakhs) | Student hours → Score (78.5%)
Weather data → Temperature (32.4°C) | Ad spend → Revenue (\$14,230)

The number can be ANY value, not from a fixed set

REAL EXAMPLE: Hospital Diabetes Prediction

The hospital wants to predict if a patient will develop diabetes.

FEATURES (Input data for each patient):

- Age (years)
- Blood Pressure (mmHg)
- BMI (body mass index)
- Glucose Level (mg/dL)
- Insulin Level
- Family History (yes/no)

LABEL (What we want to predict):

- Diabetic: YES or NO

TRAINING DATA: 768 patient records with known outcomes

HOW IT WORKS:

1. Feed 768 patients with features + known labels
2. Model finds patterns (high glucose + high BMI → likely diabetic)
3. New patient arrives → model predicts YES/NO

This is CLASSIFICATION (predicting a category: Yes/No).

If we predicted glucose LEVEL instead → that would be REGRESSION.

Classification Models: The Complete Guide

7 models every ML practitioner should know

Model	What It Does	Real-World Example	When to Use
Logistic Regression	Draws a straight line to separate two categories. Outputs a probability (0-1). Simple but effective for linearly separable data.	Predict if a customer will buy (yes/no) based on age, income, browsing time. Banks use it for loan approval.	Small dataset, 2 categories, need explainable results, quick baseline model
Decision Tree	Asks yes/no questions in sequence like a flowchart. Splits data at each node based on the most informative feature.	Doctor diagnosis: Fever? → Yes → Cough? → Yes → Duration>7 days? → Yes → Possible pneumonia	Need to EXPLAIN decisions to non-technical people, mixed data types, medium datasets
Random Forest	Builds 100+ decision trees on random subsets of data, then takes a majority vote. Reduces overfitting of single trees.	Credit card fraud detection: 500 trees each vote fraud/legit. If 400 say fraud → flag it.	Best overall accuracy needed, large datasets, don't mind black-box results
SVM (Support Vector Machine)	Finds the widest possible gap (margin) between categories. Can handle non-linear boundaries using kernel tricks.	Handwriting recognition: Separates digit '1' from '7' by finding the optimal boundary in pixel space.	Medium datasets, clear margin between classes, high-dimensional data like text
KNN (K-Nearest Neighbors)	No training phase. For new data, finds K closest known examples and picks the majority label. 'You are your neighbors.'	Movie recommendation: Your taste is similar to 5 users → recommend what they liked. Simple but slow on big data.	Small datasets, quick prototype needed, data has clear clusters, no training time budget
Naive Bayes	Uses probability theory (Bayes' theorem). Assumes features are independent. Fast and works well on text data.	Spam filter: P(spam 'free','winner','click') is very high because those words appear often in spam emails.	Text classification (spam, sentiment), very fast training needed, works with small data
CNN (Convolutional Neural Network)	Deep learning model that processes images using filters. Detects edges → shapes → objects in layers.	Google Photos face recognition: Detects faces across different angles, lighting, and ages automatically.	Image/video data, pattern recognition in spatial data, large training datasets available

Classification Decision Guide

Which model should you pick? Follow this path.

Is your data **IMAGES** or **VIDEO**?

→ Use **CNN (Convolutional Neural Network)**

CNN sees edges, shapes, textures — no other model handles raw pixels well.

Do you need to **EXPLAIN** the decision to humans?

→ Use **Decision Tree**

Shows clear IF/THEN rules. Doctors, banks, and courts need explainability.

Need the **BEST ACCURACY** and have lots of data?

→ Use **Random Forest**

500 trees voting together beat any single model. Most Kaggle competitions use this.

Small dataset with only **2 categories**?

→ Use **Logistic Regression**

Simple, fast, no overfitting risk. Great first baseline. Add complexity only if needed.

Working with **TEXT** data (emails, reviews, documents)?

→ Use **Naive Bayes**

Treats each word as independent feature. Very fast training even on large vocabularies.

Medium data, need good margins between classes?

→ Use **SVM (Support Vector Machine)**

Finds the widest gap. Kernel trick handles non-linear data. Good for text & genomics.

Regression Models: Predicting Numbers

5 models from simple to powerful

Model	What It Does	Real-World Example	When to Use
Linear Regression	Draws a straight line through data points ($y = mx + b$). Finds the best-fit line that minimizes total error.	Predict house price from size: $\text{Price} = 5000 \times \text{sqft} + 200000$. Simple, interpretable relationship.	ALWAYS try this first. Linear relationship, need baseline, explainability needed
Polynomial Regression	Fits a curved line ($y = ax^2 + bx + c$). Captures non-linear relationships that a straight line misses.	Salary vs experience: Growth is slow at start, fast in middle, plateaus at senior level. Curve fits better.	Data shows curves when plotted, linear gives poor results, relationship isn't straight
Decision Tree Regression	Splits data into regions using IF/THEN rules. Each leaf node predicts the average value of training points in that region.	Predict used car price: IF year>2020 AND km<30000 AND brand=Toyota → avg price ₹8.5L	Mixed data types, need explainability, non-linear patterns, quick results
Random Forest Regression	Builds hundreds of decision trees on random data subsets, averages their predictions. Much more stable than single tree.	Zillow/Housing.com: Predicts home values using 100+ features (size, location, age, nearby schools, crime rate).	Complex relationships, many features, best accuracy needed, large dataset
Neural Network Regression	Multiple layers of neurons learn incredibly complex patterns. Can approximate ANY mathematical function.	Stock price prediction, weather forecasting, drug response prediction — patterns too complex for other models.	Massive data (100K+ rows), very complex patterns, accuracy > explainability

REGRESSION DECISION PATH

ALWAYS start with Linear Regression → Plot residuals. If pattern is curved → try Polynomial Regression
→ If still bad or many features interact → Random Forest Regression (usually wins on tabular data)
→ If data is MASSIVE (100K+ rows) and patterns are very complex → Neural Network Regression
→ Need to EXPLAIN predictions to business stakeholders? → Decision Tree Regression

Evaluation Metrics: How Good Is Your Model?

Why accuracy alone can lie to you

ACCURACY — And Why It Lies

Accuracy = Correct / Total predictions

Sounds good, right? But consider this:

Dataset: 990 legit + 10 fraud transactions

Model says EVERYTHING is legit:

Accuracy = 990/1000 = 99%!

But it caught ZERO fraud! Useless!

This is the 'accuracy trap' with imbalanced data.

PRECISION, RECALL, F1-SCORE

PRECISION: Of all 'spam' predictions, how many were actually spam? (Don't cry wolf!)
= True Positives / (True Pos + False Pos)

RECALL: Of all actual spam emails, how many did we catch? (Don't miss any!)
= True Positives / (True Pos + False Neg)

F1-SCORE: Harmonic mean of Precision & Recall
Best single metric for imbalanced data
$$= 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

CONFUSION MATRIX — Spam Example

	Predicted: Spam	Predicted: Not Spam
Actually Spam:	85 (TP)	15 (FN)
Actually Legit:	5 (FP)	895 (TN)

TP = Correctly caught spam

FP = Legit email marked as spam

FN = Spam that slipped through

TN = Correctly allowed legit email

WHEN TO USE WHICH METRIC

PRECISION matters most when: False Positives are costly

- Spam filter (marking legit email as spam loses important messages)
- Criminal justice (convicting an innocent person is devastating)

RECALL matters most when: False Negatives are dangerous

- Cancer screening (missing a cancer case could be fatal)
- Fraud detection (missing fraud means losing money)

F1-SCORE: Use when you need balance between Precision & Recall, or when data is imbalanced

ACCURACY: Only trustworthy when classes are roughly balanced (50/50 or close)

Unsupervised Learning: All 5 Models

No labels, no teacher — find hidden patterns in data

K-MEANS CLUSTERING

What: Groups data into K clusters by minimizing distance to cluster centers. You choose K.

Use ELBOW METHOD: plot K vs error, pick the 'elbow' bend.

Example: Customer segmentation: Group 10,000 customers into 5 segments (budget, regular, premium, etc.) for targeted marketing.

When: Know how many groups you want, data is roughly spherical clusters

DBSCAN

What: Finds clusters of ANY shape based on density. Points in dense regions = cluster. Sparse points = noise/outliers. No need to specify K!

Example: Mapping hotspots: Find crime clusters in a city — some clusters are circular, others follow streets. K-Means would fail here.

When: Don't know # of groups, irregular shapes, want outlier detection automatically

ISOLATION FOREST

What: Randomly splits data with trees. Anomalies are EASY to isolate (few splits). Normal points need many splits.

Example: Credit card fraud: Isolate unusual transactions (3 AM purchase, foreign country, 10x normal amount) automatically.

When: Detect rare anomalies (<1% of data), no labeled fraud examples available

AUTOENCODER

What: Neural network that compresses data then reconstructs it. Trained on NORMAL data. Anomalies reconstruct poorly = high error.

Example: Factory quality: Train on 10,000 good product images. Defective products look 'wrong' → high reconstruction error → flagged.

When: Complex data (images, time series), need to learn 'normal' pattern, deep learning budget

PCA

What: Reduces dimensions while keeping maximum information. Projects data onto top principal components.

Example: Gene analysis: Reduce 20,000 gene features to 50 principal components. Visualize and cluster cancer types.

When: Too many features (curse of dimensionality), visualization needed, preprocessing step

DECISION: Know groups? → K-Means | Don't know? → DBSCAN | Rare events? → Isolation Forest | Learn normal? → Autoencoder | Too many features? → PCA

Reinforcement Learning: Learn by Trial & Error

No data, no labels — just rewards and penalties

AGENT

The learner/decision-maker that takes actions

ENVIRONMENT

The world the agent interacts with

REWARD

Positive feedback for good actions (+points)

PENALTY

Negative feedback for bad actions (-points)

YouTube Algorithm

Agent: Recommendation engine | Reward: User watches full video (+), User clicks away (-)
Result: Learns to suggest videos that maximize watch time. Gets better the more you use it.

Chess AI (AlphaZero)

Agent: Chess-playing program | Reward: Win (+1), Draw (0), Lose (-1)
Result: Played itself millions of games. Beat world champion engine in 4 hours of training.

Self-Driving Car

Agent: Driving AI system | Reward: Safe driving (+), Lane departure (-), Crash (--)
Result: Learns safe driving policy through simulation. Tesla uses this with real-world data.

ExamGuard (Our Project)

Agent: Monitoring system | Reward: Correctly flag cheating (+), Correctly allow legit (-penalty for false flags)
Result: Learns to distinguish cheating from normal behavior over time.

EXPLORATION vs EXPLOITATION

The core dilemma of RL:

EXPLOITATION: Keep doing what works (go to same restaurant).

EXPLORATION: Try new things (try a new restaurant — might be better or worse).

Too much exploitation = stuck in local optimum, miss better options.

Too much exploration = waste time on bad options.

Solution: Start with lots of exploration, gradually shift to exploitation (epsilon-greedy).

Three Types of ML: Side-by-Side Comparison

Supervised vs Unsupervised vs Reinforcement

Aspect	Supervised Learning	Unsupervised Learning	Reinforcement Learning
Question	WHAT is this? (classify/predict)	What PATTERNS exist?	HOW should I act?
Data Type	Labeled (input + correct answer)	Unlabeled (input only, no answers)	No data — learns from interaction
Teacher	Has a teacher with answer key	No teacher — self-discovery	Has rewards/penalties (coach)
Goal	Predict correct output	Find hidden structure	Maximize total reward
Cricket Analogy	Coach shows video of perfect shots, you copy	You watch 1000 matches, find patterns yourself	You play matches, win=reward, lose=penalty
Output	Label (spam/not) or Number (price)	Groups, anomalies, reduced features	Policy (best action for each situation)
Key Models	Logistic Reg, Random Forest, CNN	K-Means, DBSCAN, PCA	Q-Learning, Policy Gradient, DQN
Real Example	Predict if patient has diabetes	Group customers by behavior	Train robot to walk

ExamGuard USES ALL THREE TYPES!

SUPERVISED: Train face recognition model on labeled images (student A, student B, ...) to verify identity during exams

UNSUPERVISED: Cluster student behavior patterns to find unusual groups (potential organized cheating rings)

REINFORCEMENT: System learns over time which flagged behaviors are actually cheating vs false alarms, improving accuracy

Neural Networks: The Engine of Deep Learning

Inspired by the human brain — layers of connected neurons

INPUT LAYER

Receives raw data
(pixels, numbers,
words as vectors)

Neurons = # of features
No computation here
Just passes data forward

HIDDEN LAYERS

Process & transform
data through learned
weights & biases

Where learning happens
1-1000+ layers possible
More layers = 'deeper'

OUTPUT LAYER

Produces final
prediction/answer

Classification: probabilities
Regression: single number
Neurons = # of classes

ACTIVATION FUNCTIONS

Without activation: network = fancy linear equation
Activation adds NON-LINEARITY (curves!)

ReLU: $\max(0, x)$ — kills negatives, keeps positives
Most popular for hidden layers. Fast. Simple.

Sigmoid: squashes to 0-1 — good for probability
Used in binary classification output layer.

Softmax: squashes to probabilities that sum to 1
Used in multi-class output (cat/dog/bird).

FORWARD PROPAGATION

Data flows INPUT → HIDDEN → OUTPUT

Each neuron: $\text{output} = \text{activation}(\sum(\text{weight} \times \text{input}) + \text{bias})$

Layer by layer, data gets transformed

Final output = prediction

This is how the network USES what it learned

BACKPROPAGATION

Error flows BACKWARD: Output → Hidden → Input

Calculates how much each weight contributed to error

Uses chain rule of calculus (derivatives)

Updates weights to reduce error

This is how the network LEARNS

TYPES OF NEURAL NETWORKS

CNN: Convolutional — for images. Learns spatial patterns (edges, shapes, objects)

RNN/LSTM: Recurrent — for sequences. Remembers past (text, audio, time series)

Transformer: Attention mechanism — processes ALL tokens at once.
Powers GPT, BERT, Claude.

GAN: Generator + Discriminator — creates realistic fake data (images, audio)

NEURAL NETWORK TRAINING CYCLE

1. Forward pass: input → prediction | 2. Calculate loss: how wrong? | 3. Backward pass: which weights caused error? | 4. Update weights | 5. REPEAT thousands of times until loss is low (convergence)

Vectors: How Computers See the World

Everything becomes a list of numbers — that's a vector

STUDENT [20, 170, 85, 90]

Dimensions: [age, height_cm, weight_kg, score]

A student IS a point in 4D space. Similar students = nearby points. The computer doesn't see 'Usman' — it sees [20, 170, 85, 90].

HOUSE [3, 1200, 2, 10]

Dimensions: [bedrooms, sqft, bathrooms, age_years]

Every house becomes coordinates in a space. Houses with similar vectors have similar prices. ML finds the pattern: vector → price.

PHOTO [0.12, 0.87, 0.45, ... × 10,000]

Dimensions: Each number = one pixel's brightness (0=black, 1=white)

A 100×100 grayscale image = vector of 10,000 numbers. Color images have 3 values per pixel (RGB) → 30,000 numbers.

EMAIL [7, 4, 1, 1, 0, 1]

Dimensions: [word_count_hundreds, links, has_attachment, has_'free', has_'urgent', from_unknown]

An email becomes a feature vector. Spam emails cluster together in this space (many links, unknown sender, 'free' keyword).

CRICKET PLAYER [274, 13906, 52.7, 46]

Dimensions: [matches, runs, average, centuries]

Sachin = [463, 18426, 44.8, 49]. Kohli = [274, 13906, 52.7, 46]. Dot product tells us how similar they are!

KEY: ML doesn't understand 'houses' or 'students'. It only sees vectors (lists of numbers). Our job: convert real-world things into good vectors.

Dot Product: Measuring Similarity

Multiply matching positions, add them up — higher = more similar

NETFLIX EXAMPLE: Movie Ratings (1-5 stars)

Action Romance Comedy Horror

You = [5, 1, 3, 4]

User B = [4, 2, 3, 5]

User C = [1, 5, 4, 1]

You love Action & Horror, hate Romance.

User B also loves Action & Horror!

User C loves Romance — opposite of you.

RESULT: Netflix Recommends!

You vs User B = 51 (SIMILAR taste!)

You vs User C = 26 (DIFFERENT taste)

Netflix says: 'User B loved Interstellar
and you haven't seen it yet...'

RECOMMENDATION: Watch Interstellar!
(Because User B's taste matches yours)

THE MATH: Multiply matching positions, sum up

You · User B = $(5 \times 4) + (1 \times 2) + (3 \times 3) + (4 \times 5)$

= 20 + 2 + 9 + 20

= 51 ← HIGH similarity!

You · User C = $(5 \times 1) + (1 \times 5) + (3 \times 4) + (4 \times 1)$

= 5 + 5 + 12 + 4

= 26 ← LOW similarity!

WHY DOT PRODUCT WORKS FOR SIMILARITY

When both users rate a genre HIGH → big positive product

$(5 \times 4 = 20 \text{ for Action})$ ← both love it!

When one rates HIGH and other LOW → small product

$(1 \times 5 = 5 \text{ for Romance})$ ← disagreement

When both rate LOW → small product (doesn't help)

Sum of all these = overall similarity score

DOT PRODUCT IS EVERYWHERE IN ML

Every neuron in a neural network computes a dot product: $\text{output} = \text{dot}(\text{weights}, \text{inputs}) + \text{bias}$ | Word similarity in LLMs: dot product of word vectors | Attention mechanism in Transformers: dot product of query & key vectors

Derivatives: The Rate of Change

How fast is something changing? That's the derivative.

SAMOSAS SHOP EXAMPLE

You sell samosas. You change the PRICE and see how SALES change.

Price ₹10 → Sell 100/day

Price ₹12 → Sell 80/day

Price ₹15 → Sell 50/day

Rate of change (derivative):

From ₹10→₹12: $\Delta \text{sales} / \Delta \text{price} = -20/2 = -10$

'For every ₹1 increase, you lose 10 sales'

This RATE OF CHANGE is the derivative!

It tells you the DIRECTION and SPEED of change.

ML EXAMPLE: House Price Model

Model: Price = weight × size + bias

Current weight = 0.01 (per sqft)

House: 1000 sqft, actual price = ₹50 Lakhs

Model prediction: $0.01 \times 1000 + 10 = ₹20$ Lakhs

Error = $(20 - 50)^2 = 900$

Derivative of error with respect to weight:

$d(\text{error})/d(\text{weight}) = -60,000$

What this means in plain English:

'The weight for size is TOO LOW!'

'Increase it to reduce the error'

'Specifically, increase by ~0.008'

WHY DERIVATIVES MATTER IN ML — The Big Picture

1. We make a PREDICTION (forward pass)
2. We calculate the ERROR (loss function): how wrong were we?
3. We take the DERIVATIVE of the error with respect to each weight
→ This tells us: should this weight go UP or DOWN? By how much?
4. We UPDATE the weight in the direction that REDUCES error
5. REPEAT thousands of times

Without derivatives, we'd have to guess randomly which direction to adjust weights.

Derivatives give us the EXACT direction and magnitude to move — like a compass pointing to lower error.

Positive derivative → weight is too high → decrease it | Negative derivative → weight is too low → increase it | Zero derivative → perfect! (or stuck at local minimum)

Gradient Descent: Finding the Best Weights

The algorithm that makes neural networks learn

ANALOGY: Blindfolded on a Hill

You're blindfolded on a hilly landscape.
Goal: reach the LOWEST point (valley).

You can't SEE, but you can FEEL:

- Which direction slopes downward
- How steep the slope is

Strategy: Take a step in the steepest downward direction. Repeat.

That's EXACTLY what gradient descent does with weights and error!

WORKED EXAMPLE: Weight Update

Current weight = 0.5

Learning rate (step size) = 0.1

Gradient (slope) = 2.0

Formula: $\text{new_weight} = \text{old} - \text{LR} \times \text{gradient}$

$\text{new_weight} = 0.5 - 0.1 \times 2.0$

$\text{new_weight} = 0.5 - 0.2$

$\text{new_weight} = 0.3$

The weight decreased from 0.5 to 0.3
because the gradient said 'you're too high,
go down!'

LEARNING RATE: The Step Size

TOO BIG (LR = 1.0):

Takes huge steps → overshoots the minimum → bounces around → never converges. Like running downhill.

JUST RIGHT (LR = 0.01-0.1):

Steady steps → reliably reaches minimum → stable training.

TOO SMALL (LR = 0.0001):

Tiny steps → takes forever to reach minimum → training never finishes.

LOCAL MINIMA: The Trap

Gradient descent follows the slope DOWN. But what if there are multiple valleys? It might get stuck in a SHALLOW valley (local minimum) instead of reaching the DEEPEST valley (global minimum).

Solutions used in practice:

- Momentum: Like a ball rolling — carries speed to escape shallow valleys
- Adam optimizer: Adapts learning rate per weight (most popular today)
- Random restarts: Try from multiple starting points
- Batch size variation: Different data subsets → different gradients

In practice with neural networks, local minima are usually 'good enough' — the loss landscape has many valleys of similar depth.

GRADIENT DESCENT VARIANTS

BATCH GD: Use ALL data to compute gradient each step.
Most accurate gradient. Very slow on large datasets.

STOCHASTIC GD (SGD): Use ONE random sample per step.
Noisy but fast. Can escape local minima due to noise.

MINI-BATCH GD: Use small batch (32-256 samples) per step.
Best of both worlds. This is what everyone actually uses.

ADAM: Mini-batch + adaptive learning rate + momentum.
The default optimizer in modern deep learning.
Works well 'out of the box' on most problems.

Train/Test Split, Overfitting & Underfitting

Why we split data and what goes wrong

80/20 TRAIN-TEST SPLIT

1000 data points → 800 train + 200 test

TRAINING SET (80%): Model learns from this.

Sees features AND correct answers.

Adjusts weights to minimize error.

TEST SET (20%): Model is TESTED on this.

Never seen during training!

Measures real-world performance.

Like studying from practice questions,
then taking a DIFFERENT final exam.

OVERFITTING — Memorized, Not Learned

Train accuracy: 99%

Test accuracy: 30% ← HUGE gap!

The model MEMORIZED the training data
including noise and outliers, instead of
learning the general pattern.

Like a student who memorizes answers
to practice questions but can't solve
new problems on the exam.

Fixes: More data, simpler model, dropout,
regularization, early stopping

UNDERFITTING — Too Simple

Train accuracy: 50%

Test accuracy: 48% ← Both low!

The model is TOO SIMPLE to capture
the patterns in the data.

Like using a straight line to fit data
that clearly curves — it can't learn
the real relationship.

Like a student who only studied
chapter 1 for a 10-chapter exam.

Fixes: More complex model, more features

HOW DO YOU TEST DIFFERENT ML TYPES?

SUPERVISED LEARNING — Automatic evaluation (easiest!)

Split data 80/20 → Train on 80% → Predict on 20% → Compare predictions vs actual labels → Get accuracy/F1/etc.

You know the RIGHT ANSWERS, so you can measure exactly how well the model performs.

UNSUPERVISED LEARNING — Human evaluation needed (hardest!)

No labels to compare against! How do you know if clusters are 'correct'?

Methods: Silhouette score (math check), domain expert review, downstream task performance

Example: Show customer segments to marketing team → 'Do these groups make business sense?'

REINFORCEMENT LEARNING — Performance in environment

Let agent play the game/drive the car/manage the system → Measure total reward over time

Compare against: random policy, human performance, previous version of the agent

Example: Chess AI wins 95% of games against previous version → improvement confirmed

Full Training Cycle: Putting It All Together

House price prediction — 6 steps with real numbers

STEP 1: VECTORIZE

House = [1000 sqft, 5 km from city]
Vector: [1000, 5]
Weights: [0.01, -2.0], Bias: 10

STEP 2: PREDICT (Forward Pass)

Price = $(0.01 \times 1000) + (-2.0 \times 5) + 10$
Price = $10 - 10 + 10 = ₹20.05$ Lakhs
But actual price = ₹50 Lakhs!

STEP 3: CALCULATE LOSS

Loss = $(\text{predicted} - \text{actual})^2 = (20.05 - 50)^2$
Loss = $(-29.95)^2 = 897$
This is MSE — Mean Squared Error

STEP 4: DERIVATIVES

$d(\text{loss})/d(\text{size_weight}) = -59,900$
Meaning: 'Size weight is WAY too low!'
 $d(\text{loss})/d(\text{distance_weight}) = 299.5$
Meaning: 'Distance weight is slightly too negative'

STEP 5: UPDATE WEIGHTS

LR = 0.0001
size_weight: $0.01 - (0.0001 \times -59900) = 0.01 + 5.99 \rightarrow$
clipped to 0.018
dist_weight: $-2.0 - (0.0001 \times 299.5) = -2.03$
Weights moved toward BETTER values!

STEP 6: REPEAT × 500

After 500 iterations:
Weights: [0.042, -1.5], Bias: 12
Prediction: $(0.042 \times 1000) + (-1.5 \times 5) + 12 = ₹46.5\text{L}$
Much closer to ₹50L! MODEL IS TRAINED!

CONVERGENCE

When loss stops decreasing significantly between iterations, the model has 'converged'.
Training is complete. Typical: 100-10,000 iterations depending on complexity.

5 THINGS TO REMEMBER

1. Everything is vectors 2. Forward pass = prediction 3. Loss = how wrong 4. Derivatives = direction to fix 5. Gradient descent = actually fixing weights

Key Terms Reference: 25 Terms You Must Know

Quick-reference glossary for all Foundations concepts

AI: Making machines that can think, learn, decide	Feature: Input variable (age, size, pixels)	Gradient Descent: Algorithm that minimizes loss by following slope
ML: Machines learn from data instead of explicit rules	Label: Correct answer in training data	Learning Rate: Step size for weight updates (too big/small = bad)
Deep Learning: ML with multi-layer neural networks	Vector: List of numbers representing something	Overfitting: Memorized training data, fails on new data
LLM: Large language model (GPT, Claude) using Transformers	Dot Product: Multiply matching positions & sum — measures similarity	Underfitting: Model too simple, fails on all data
Supervised: Learn from labeled data (input + answer)	Neural Network: Layers of neurons that learn patterns from data	Train/Test Split: 80% learn, 20% evaluate real performance
Unsupervised: Find patterns in unlabeled data	Forward Pass: Data flows input→output to make prediction	Activation Function: Adds non-linearity (ReLU, Sigmoid, Softmax)
Reinforcement: Learn by trial, error, rewards & penalties	Backpropagation: Error flows backward to update weights	Convergence: When loss stops decreasing — training is done
Classification: Predict a category/label (spam or not)	Loss Function: Measures how wrong the prediction is	
Regression: Predict a continuous number (house price)	Derivative: Rate of change — tells direction to adjust	